

COMPLETE PROJECT DOCUMENTATION

ExperimentHome: Remote Ohm's Law Laboratory

A Cross-Platform Mobile Application for Monitoring and Controlling
Remote Lab Experiments via IoT

Author: Ekane Clinton Ntongwe

Student ID: FE22A198

Institution: University of Buea

Department: Computer Engineering

Level: Final Year (400 Level)

Academic Year: 2025 – 2026

GitHub Repository: <https://github.com/ClintonEkane/experimenthome>

*NOTE TO AI: This document contains everything about the ExperimentHome project.
Use it to write the complete final year dissertation report.
All technical details, design decisions, challenges, and solutions are documented herein.*

1. PROJECT OVERVIEW

1.1 Project Title

Design and Implementation of a Cross-Platform Mobile Application for Monitoring and Controlling Remote Lab Experiments.

1.2 Problem Statement

Traditional laboratory experiments in engineering and science education require physical presence in a dedicated laboratory facility. This creates significant barriers for students who face geographical distance from campus, limited laboratory opening hours, financial constraints on transport, or situations such as pandemic lockdowns that prevent physical attendance. In the Department of Computer Engineering at the University of Buea, students are required to perform practical experiments — including the fundamental Ohm's Law verification experiment — as part of their coursework. Without access to the physical laboratory, these students cannot complete their practicals, which directly impacts their academic performance and understanding of core engineering concepts.

The problem is therefore: How can a student perform a real hardware-based laboratory experiment (specifically Ohm's Law: $I = V/R$) from any location using only a smartphone, without requiring physical access to the laboratory equipment?

1.3 Proposed Solution

ExperimentHome is a cross-platform mobile application (built with Flutter/Dart) that allows a student to remotely control physical laboratory hardware installed at a fixed lab station. The student can: select a resistor value, start and stop the experiment, observe live current measurements, view a real-time graph of current vs. time, and review past session records — all from their smartphone, from any location with internet access.

The physical hardware (ESP32 microcontroller, ACS712 current sensor, 4-channel relay module, resistors, and an LED) is permanently installed at the lab station. The mobile app communicates with this hardware over the internet using the MQTT protocol. User authentication and station-access management are handled by Google Firebase.

1.4 Project Objectives

- Design and implement a Flutter mobile application (Android and iOS) for remote experiment control.
- Design and implement ESP32 firmware for relay switching and current sensing.
- Establish real-time bidirectional communication between the app and hardware using MQTT over TLS.
- Implement a station-locking mechanism to prevent simultaneous access by multiple users.
- Display real-time current readings and a live scrolling graph on the mobile app.
- Record experiment session data (resistor, peak current, timestamps) in Firebase Firestore.

- Demonstrate Ohm's Law ($I = V/R$) using four different resistor values: 100 Ω , 220 Ω , 470 Ω , and 1 k Ω .
- Ensure the system handles all app-close scenarios (back button, home button, swipe-away) gracefully.

1.5 Project Scope

The project is scoped to a single experiment type (Ohm's Law) with one physical hardware station. The architecture is designed to scale to multiple experiments and stations, but only one station is implemented for this dissertation. The mobile app targets Android (primary test device: Samsung SM-G781U running Android 13) and is built with Flutter for cross-platform compatibility. The hardware station uses a 5V power supply, an ACS712 20A current sensor, a four-channel relay module, and four resistors in the range 100 Ω to 1 k Ω .

2. TECHNOLOGY STACK AND JUSTIFICATION

The following technologies were selected for ExperimentHome. Each choice is justified against alternative options considered.

Technology	Role	Justification
Flutter / Dart	Cross-platform mobile application	Single codebase for Android and iOS; reactive UI; strong async/stream support; hot reload for rapid iteration.
ESP32 DevKit V1	Embedded hardware controller	Built-in Wi-Fi; dual-core 240 MHz processor; 12-bit ADC; Arduino ecosystem with mature MQTT and JSON libraries; low cost.
Arduino (C++)	ESP32 firmware language	PubSubClient (MQTT), WiFiClientSecure (TLS), ArduinoJson libraries are production-ready and well-documented.
MQTT over TLS	Real-time IoT messaging	Publish-subscribe model ideal for IoT; minimal overhead vs HTTP; broker handles routing; supports Last Will & Testament; QoS levels.
HiveMQ Cloud	MQTT broker (hosted)	Managed cloud broker; free tier; supports TLS on port 8883; reliable uptime; no self-hosting complexity.
Firebase Authentication	User identity management	Ready-made email/password auth; no custom auth server needed; integrates natively with Flutter via FlutterFire.
Cloud Firestore	Station locking and session history	Real-time listeners (no polling); NoSQL document model fits the schema; FieldValue.delete() supports our lock-release strategy.
Provider (Flutter package)	State management / DI	Lightweight; fits the service-layer architecture; allows clean separation of business logic from UI.
mqtt_client (Dart package)	MQTT client for Flutter	Supports TLS; async message streams; well-maintained; supports MQTT 3.1 and 3.1.1.

Table 2.1: Technology Stack and Justification

2.1 Technologies Considered but Not Used

Technology	Considered For	Reason Not Used
HTTP REST API	App-to-hardware communication	Polling-based; high latency for real-time sensor data; ESP32 as HTTP server is fragile under concurrent connections.
WebSocket	Real-time communication	More complex to implement on ESP32; MQTT is the established IoT standard with better tooling.
React Native	Mobile app framework	Flutter chosen for better performance and tighter Dart/stream integration.
AWS IoT Core	MQTT broker	More complex setup; pricing less predictable; HiveMQ free tier sufficient for this project.
INA219 (I2C)	Current sensing	Would be a better sensor for mA-level measurements, but project uses ACS712 which was already procured.
SQLite (local)	Session storage	Firestore chosen for cloud sync and real-time capabilities across devices.

Table 2.2: Technologies Considered but Not Selected

3. HARDWARE DESIGN

3.1 Components List

Component	Specification	Quantity	Role
ESP32 DevKit V1	Dual-core 240 MHz, Wi-Fi, 12-bit ADC, 3.3V logic	1	Main microcontroller
ACS712 Current Sensor	20A version, sensitivity = 100 mV/A, Vout_zero = 2.5 V	1	Measures current through circuit
4-Channel Relay Module	Active-LOW, 5V coil, 10A contacts, opto-isolated	1	Switches resistors in/out of circuit
Resistor 100 Ω	0.25 W, 5% tolerance	1	Resistor option 1 (Relay 1)
Resistor 220 Ω	0.25 W, 5% tolerance	1	Resistor option 2 (Relay 2)
Resistor 470 Ω	0.25 W, 5% tolerance	1	Resistor option 3 (Relay 3)
Resistor 1 k Ω	0.25 W, 5% tolerance	1	Resistor option 4 (Relay 4)
Red LED	Standard 5 mm, ~2 V forward voltage	1	Visual load — lights when current flows
5V Power Supply	USB adapter or bench PSU	1	Powers relay module and circuit
Breadboard + Jumper Wires	-	As needed	Circuit assembly

Table 3.1: Hardware Components

3.2 Circuit Description

3.2.1 Current Path

The main current measurement path is: 5V Power Supply → ACS712 IP+ (current input) → ACS712 IP- (current output) → 4-channel relay module COM terminal → Relay NO terminal (when relay is closed) → Selected Resistor → LED (red, acts as load) → GND. Only one relay is ever closed at a time, so only one resistor is in the circuit at any moment.

3.2.2 ESP32 GPIO Connections

ESP32 Pin	Connected To	Direction	Notes
GPIO 25 (D25)	Relay Module IN1	Output	Controls relay 1 (100 Ω). LOW = relay closed.
GPIO 26 (D26)	Relay Module IN2	Output	Controls relay 2 (220 Ω). LOW = relay closed.
GPIO 27 (D27)	Relay Module IN3	Output	Controls relay 3 (470 Ω). LOW = relay closed.
GPIO 14 (D14)	Relay Module IN4	Output	Controls relay 4 (1 k Ω). LOW = relay closed.
GPIO 34 (D34)	ACS712 VOUT	ADC Input	12-bit ADC, input-only pin. Reads 0–3.3V from sensor.
GPIO 2	Built-in Blue LED	Output	HIGH = LED on. Indicates fully connected to broker.
5V (VIN)	Relay Module VCC	Power	5V from USB powers relay coil.

3.3V	ACS712 VCC	Power	Sensor powered from 3.3V rail.
GND	Relay Module GND, ACS712 GND, Circuit GND	Power	Common ground for all components.

Table 3.2: ESP32 GPIO Pin Connections

3.2.3 Relay Module Wiring Detail

The relay module used is an active-LOW module. This means the relay coil is energised (relay closes, connecting COM to NO) when the IN pin is driven LOW. When the IN pin is HIGH (or floating), the relay is open (COM is disconnected from NO). IMPORTANT: All relay IN pins are initialised to HIGH at boot. This ensures no relay closes during the ESP32 boot sequence, preventing unintended current flow and the relay-clicking/LED-blinking artefact that occurs when GPIO pins float before setup() runs.

Relay internal connections for this project: Each relay's COM terminal is wired to the ACS712 IP– output. Each relay's NO terminal is wired to one resistor. Each relay's NC terminal is left unconnected. All four resistors share a common output node that connects to the LED anode. The LED cathode connects to GND.

3.2.4 ACS712 Current Sensor

The ACS712 is a Hall-effect based current sensor. The 20A version (used in this project) has the following key characteristics: Zero-current output voltage = 2.5 V (exactly half of 5V supply, but sensor is powered at 3.3V so the output is read via the ESP32 ADC which is also 3.3V reference). Sensitivity = 100 mV/A (i.e., 0.1 V per ampere of current). For positive current (flowing IP+ to IP–), the output voltage rises above 2.5V. For negative current (flowing IP– to IP+), the output voltage falls below 2.5V. In this project, current always flows in the positive direction (IP+ to IP–), so the output is always $\geq 2.5\text{V}$ when current is flowing.

3.2.5 Known Hardware Limitation

IMPORTANT: The ACS712-20A is designed for currents of 1A to 20A. The experiment currents for this project range from approximately 3 mA (1 k Ω resistor) to 50 mA (100 Ω resistor) assuming a 5V supply. These currents are at or below the noise floor of the 20A sensor (which has significant ADC noise at the milliamp level). The theoretical voltage change for 50 mA is only 5 mV (50 mA $\times$ 100 mV/A = 5 mV), which is within the ADC noise floor of the ESP32. The recommended upgrade for future versions is the ACS712-05A (5A version, 185 mV/A sensitivity) or the INA219 (I2C precision current sensor, accurate to microamps). Despite this limitation, the system correctly demonstrates the proportional relationship between resistance and current, which is the core educational objective.

3.3 Expected Measurement Values (Ohm's Law)

Resistor (Ω)	Relay	GPIO	Expected Current ($I = V/R$, $V = 5\text{V}$)	Voltage change on ACS712 output
100	Relay 1	GPIO 25	50.0 mA	5.0 mV above 2.5V

220	Relay 2	GPIO 26	22.7 mA	2.3 mV above 2.5V
470	Relay 3	GPIO 27	10.6 mA	1.1 mV above 2.5V
1000	Relay 4	GPIO 14	5.0 mA	0.5 mV above 2.5V

Table 3.3: Expected Current Values (Ohm's Law: $I = V/R$, $V = 5V$)

4. SYSTEM ARCHITECTURE

4.1 Three-Layer Architecture Overview

The ExperimentHome system is structured as a three-layer (three-tier) architecture. Each layer is independent and communicates with adjacent layers through well-defined interfaces (MQTT topics, Firebase SDK). This separation allows each layer to be modified or replaced without impacting the others.

Layer 1 — Hardware Layer:

The physical ESP32 hardware station located in the laboratory. It consists of the ESP32 microcontroller, ACS712 current sensor, 4-channel relay module, resistors, and LED. The ESP32 firmware controls all hardware operations and communicates with the outside world via Wi-Fi and MQTT.

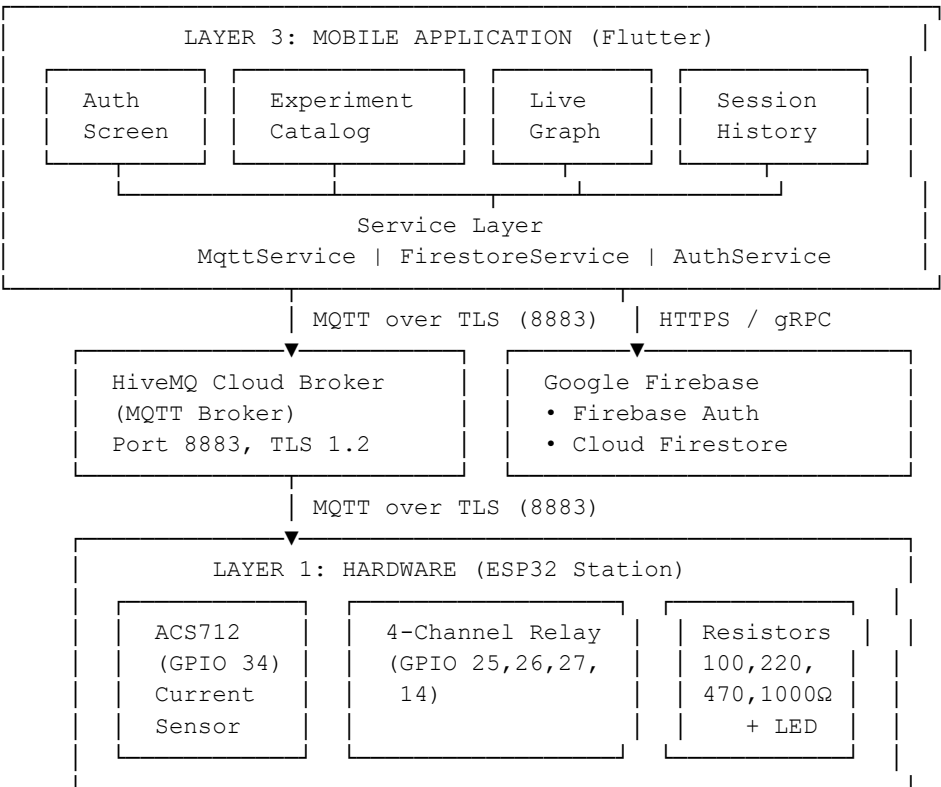
Layer 2 — Communication and Cloud Backend Layer:

Two cloud services form this layer. The HiveMQ Cloud MQTT broker routes real-time messages (sensor data, control commands) between the mobile app and the ESP32. Google Firebase provides user authentication (Firebase Auth) and persistent data storage (Cloud Firestore) for station locks and session records.

Layer 3 — Application Layer:

The Flutter mobile application running on the student's smartphone. It provides the user interface, implements the business logic (station locking, session management), and communicates with both the MQTT broker and Firebase.

4.2 Architecture Diagram (Text Representation)



4.3 Data Flow

Two data flows operate simultaneously during an experiment session:

- **Command Flow (App → ESP32):** The app publishes a JSON command to the MQTT control topic. The HiveMQ broker routes it to the ESP32, which has subscribed to that topic. The ESP32 parses the command and actuates the relays.
- **Sensor Flow (ESP32 → App):** Every second while the experiment is running, the ESP32 reads the ACS712, computes the current in mA, and publishes a JSON message to the MQTT current topic. The app receives this via its subscription, updates the display and graph.
- **Authentication Flow (App → Firebase):** On login/register, Firebase Auth returns a user session. The app uses the session UID for all Firestore writes (station lock, session records).
- **Station Lock Flow (App → Firestore):** When the app enters the experiment screen, it writes the station lock to Firestore. Other devices' real-time listeners detect this change and update their availability displays. When the app exits, the lock is cleared.

5. ESP32 FIRMWARE

5.1 Firmware File

File: esp32_firmware/esp32_firmware.ino

The firmware is written in C++ using the Arduino framework. It uses the following libraries: WiFi.h (built-in), WiFiClientSecure.h (TLS support), PubSubClient.h (MQTT), ArduinoJson.h (JSON serialisation/deserialisation).

5.2 Configuration Constants

```
WiFi SSID:          "clinton"
WiFi Password:      "clinton1"
MQTT Broker:        147bea8727e745e8993c3928a1f4199f.s1.eu.hivemq.cloud
MQTT Port:          8883 (TLS)
MQTT Username:      "experimenthome"
MQTT Password:      "Experimenthome1"
MQTT Client ID:     "esp32_ohmslaw_station1"
```

```
GPIO Pins:
Relay 1 (100Ω) : GPIO 25
Relay 2 (220Ω) : GPIO 26
Relay 3 (470Ω) : GPIO 27
Relay 4 (1kΩ)  : GPIO 14
ACS712 ADC     : GPIO 34
Blue LED       : GPIO 2
```

```
ACS712 Constants:
Sensitivity      : 100.0 mV/A (20A version)
Zero Voltage     : 2.5 V
ADC Samples      : 100 (averaged per reading)
```

```
Timing:
Heartbeat interval      : 5000 ms
Current publish interval: 1000 ms (only when experiment running)
```

5.3 MQTT Topics

Topic	Publisher	Subscriber	Payload	Purpose
experiments/ohms-law/stations/station-1/control	Flutter App	ESP32	{"action":"start"} {"action":"stop"} {"action":"select_resistor","value":220}	Control commands from app to hardware
experiments/ohms-law/stations/station-1/current	ESP32	Flutter App	{"current_mA":14.3,"timestamp":1234}	Live current reading every 1 second (only when running)
experiments/ohms-law/stations/station-1/status	ESP32	Flutter App	{"online":true,"timestamp":1234}	Heartbeat every 5 seconds
experiments/ohms	HiveMQ	Flutter	{"online":false,"reason":"lwt"}	Auto-

- law/stations/station-1/session	Broker (LWT)	App		published by broker when ESP32 disconnects ungracefully
-------------------------------------	-----------------	-----	--	--

Table 5.1: MQTT Topics

5.4 setup() Function

The setup() function runs once on boot. It performs the following in order:

- Initialise serial port at 115200 baud for debugging.
- Set LED pin (GPIO2) as OUTPUT, drive LOW (LED off).
- Set all four relay pins as OUTPUT, drive HIGH (all relays open — active LOW).
- Set ADC resolution to 12 bits (0–4095 range).
- Call connectWifi() — blocks until Wi-Fi is established.
- Call connectMqtt() — registers LWT, blocks until MQTT broker connection is established, then subscribes to control topic and drives LED HIGH (steady on = fully connected).

5.5 loop() Function

The loop() function runs continuously after setup(). It performs:

- Check if MQTT client is still connected. If not, call connectMqtt() to reconnect.
- Call mqttClient.loop() to process incoming messages (triggers onMqttMessage callback for any received control command).
- If 5 seconds have elapsed since last heartbeat: call publishHeartbeat() which sends {"online":true, "timestamp":<seconds>} to the status topic.
- If experiment is running AND 1 second has elapsed since last current publish: call publishCurrent() which reads ACS712, computes mA, and sends {"current_mA":<value>, "timestamp":<seconds>} to the current topic.

5.6 Current Measurement (readCurrentMa)

```
float readCurrentMa() {
    long sum = 0;
    for (int i = 0; i < 100; i++) {
        sum += analogRead(ACS712_PIN);    // GPIO 34, 12-bit (0-4095)
        delayMicroseconds(100);           // 100 µs between samples
    }
    float avgAdc = sum / 100.0;
    float voltage = (avgAdc / 4095.0) * 3.3;    // Convert to volts
    float currentA = (voltage - 2.5) / (100.0/1000.0); // I = (V - V_zero)
    / sensitivity
    return max(0.0f, currentA * 1000.0);        // Clamp negative
    noise to 0, return mA
}
```

5.7 MQTT Command Handler (onMqttMessage)

```

void onMqttMessage(char* topic, byte* payload, unsigned int length) {
    StaticJsonDocument<128> doc;
    if (deserializeJson(doc, payload, length)) return;
    const char* action = doc["action"];
    if (!action) return;

    if (strcmp(action, "start") == 0) {
        experimentRunning = true;
        closeRelay(activeRelayIndex);          // Close the currently selected
relay

    } else if (strcmp(action, "stop") == 0) {
        experimentRunning = false;
        openAllRelays();                      // Open all relays – stops
current flow

    } else if (strcmp(action, "select_resistor") == 0) {
        int value = doc["value"];
        for (int i = 0; i < 4; i++) {
            if (RESISTOR_VALUES[i] == value) {
                activeRelayIndex = i;
                if (experimentRunning) closeRelay(i); // Switch relay if
experiment active
                break;
            }
        }
    }
}

```

5.8 Relay Control Functions

```

// Active-LOW module: HIGH = relay open, LOW = relay closed

void openAllRelays() {
    for (int i = 0; i < 4; i++)
        digitalWrite(RELAY_PINS[i], HIGH); // HIGH = open
}

void closeRelay(int index) {
    openAllRelays(); // First ensure all are open
    digitalWrite(RELAY_PINS[index], LOW); // Then close only the target
relay
// Ensures only ONE relay ever
closes
}

```

5.9 Boot Behaviour

IMPORTANT: All relay pins are driven HIGH immediately in `setup()`, BEFORE connecting to Wi-Fi or MQTT. This is critical because GPIO pins float during the boot sequence (before `setup()` runs), which on active-LOW relay modules can momentarily close relays, causing the relay to click and the LED to flicker. Driving all pins HIGH as the first action in `setup()` minimises this window. The `closeRelay(0)` default that was previously in `setup()` has been removed — all relays remain open until the user explicitly sends a "start" command via MQTT.

6. FLUTTER MOBILE APPLICATION

6.1 App Metadata

```
App Name:           ExperimentHome
Package Name:       com.remotelab.experimenthome
Framework:         Flutter (Dart)
Min Android SDK:    21 (Android 5.0 Lollipop)
Target SDK:         Android 13 (API 33)
Test Device:        Samsung SM-G781U (RFCR71S4HLM), Android 13
GitHub:             https://github.com/ClintonEkane/experimenthome
```

6.2 Project File Structure

```
experimenthome/
├── lib/
│   ├── core/
│   │   ├── constants/
│   │   │   └── app_colors.dart           # Color palette
│   │   ├── models/
│   │   │   ├── experiment.dart           # Experiment + Station models
│   │   │   ├── current_reading.dart      # CurrentReading model
│   │   │   └── session_record.dart       # SessionRecord model
│   │   └── services/
│   │       ├── auth_service.dart         # Firebase Auth wrapper
│   │       ├── firestore_service.dart    # Firestore CRUD
│   │       └── mqtt_service.dart         # MQTT connection + streams
│   ├── features/
│   │   ├── auth/
│   │   │   └── screens/
│   │   │       └── login_screen.dart
│   │   ├── catalog/
│   │   │   └── screens/
│   │   │       └── catalog_screen.dart    # Experiment list
│   │   └── experiment/
│   │       ├── screens/
│   │       │   └── experiment_screen.dart # Main experiment UI
│   │       └── widgets/
│   │           ├── resistor_selector.dart
│   │           ├── live_graph.dart
│   │           ├── status_indicator.dart
│   │           └── current_display.dart
│   └── main.dart                         # App entry point, Provider
├── setup
├── android/
│   └── app/src/main/kotlin/.../
│       └── MainActivity.kt               # onDestroy force-kill
└── esp32_firmware/
    └── esp32_firmware.ino
```

6.3 Key Data Models

6.3.1 Station Model (*experiment.dart*)

```
class Station {
  final String id;           // e.g., "station-1"
  final String? lockedBy;    // UID of locking user (null =
available)
  final String? lockedByName; // Display name
  final DateTime? lockedSince; // When lock was acquired
```

```

// True if lock is older than 10 minutes
bool get isStale => lockedSince != null &&
    DateTime.now().difference(lockedSince!).inMinutes > 10;

// True if no lock, or lock is stale
bool get isAvailable => lockedBy == null || isStale;

// True if no lock, own lock, or stale lock
bool isAvailableFor(String userId) =>
    lockedBy == null || lockedBy == userId || isStale;
}

```

6.3.2 Firestore Schema

```

// Station document (station lock):
experiments/{experimentId}/stations/{stationId}
  lockedBy:      String?    // null when available
  lockedByName:  String?    // null when available
  lockedSince:   int?       // epoch milliseconds, null when available

// Session document:
users/{userId}/sessions/{autoId}
  experimentId:  String
  stationId:     String
  resistorOhms:  int
  peakCurrentMa: double
  startedAt:     Timestamp
  endedAt:       Timestamp  // written when session ends

```

6.4 FirestoreService — Station Lock Functions

***lockStation()* — Used when entering experiment screen or resuming from background**

```

Future<void> lockStation({required String experimentId,
    required String stationId, required String userId,
    required String displayName}) async {
  await _db
    .collection("experiments").doc(experimentId)
    .collection("stations").doc(stationId)
    .set({
      "lockedBy": userId,
      "lockedByName": displayName,
      "lockedSince": DateTime.now().millisecondsSinceEpoch,
    }, SetOptions(merge: true));
}

```

***unlockStation()* — Used when back button is pressed (sets fields to null)**

```

Future<void> unlockStation(String experimentId, String stationId) async {
  await _db
    .collection("experiments").doc(experimentId)
    .collection("stations").doc(stationId)
    .set({"lockedBy": null, "lockedByName": null, "lockedSince": null},
        SetOptions(merge: true));
}

```

***deleteStationLock()* — Used when app is closed/swiped away (deletes fields entirely)**

```

Future<void> deleteStationLock(String experimentId, String stationId)
  async {
    try {

```

```

        await _db
            .collection("experiments").doc(experimentId)
            .collection("stations").doc(stationId)
            .update({
                "lockedBy": FieldValue.delete(),
                "lockedByName": FieldValue.delete(),
                "lockedSince": FieldValue.delete(),
            });
    } catch (_) {}
}

```

DESIGN DECISION — null vs delete: When the back button is pressed, the lock fields are set to null (unlockStation). The document structure is preserved. When the app is closed/swiped away, the lock fields are deleted entirely (deleteStationLock with FieldValue.delete()). The delete approach is used for app-close because it is more aggressive and leaves no stale data. Both approaches result in the station being available (lockedBy == null OR field absent), but deleteStationLock is the "emergency release" path.

6.5 ExperimentScreen — Lifecycle Management

The ExperimentScreen is the most complex screen. It implements WidgetsBindingObserver to detect app lifecycle changes and handles all station lock release scenarios.

6.5.1 State Variables

```

Station? _myStation;           // The station this user has locked
bool _claiming = true;         // True while initial claim is in progress
bool _lockReleased = false;    // Guards against releasing the lock twice
bool _sessionActive = false;    // True while experiment is running
int _selectedResistor = 100;    // Currently selected resistor value
double _peakCurrentMa = 0;     // Peak current in this session
String? _sessionDocId;         // Firestore ID of current session record
DeviceStatus _deviceStatus;    // online / offline indicator for UI
List<CurrentReading> _readings = []; // Last 120 readings for graph

```

6.5.2 _init() — Station Claim on Screen Open

```

Future<void> _init() async {
    Station? toReclaim;
    for (final s in widget.stations) {
        if (s.lockedBy == user.uid || s.isStale) {
            toReclaim ??= s; // Track first station to reclaim
            await _firestore.deleteStationLock(widget.experiment.id, s.id);
        }
    }
    // CRITICAL: Cannot use widget.stations after delete – it is a stale
    // snapshot.
    // Instead use toReclaim (the station reference we already have).
    if (toReclaim != null) {
        await _claimStation(toReclaim); // Claim the freed station directly
    } else {
        await _tryClaimStation();        // Find available station from
        snapshot
    }
}

```

6.5.3 Lifecycle State Handler


```

@override
void didChangeAppLifecycleState(AppLifecycleState state) {
  if (state == AppLifecycleState.paused ||
      state == AppLifecycleState.hidden ||
      state == AppLifecycleState.detached) {
    _closeSession(); // App going to background or being killed
  } else if (state == AppLifecycleState.resumed) {
    _reclaimOnResume(); // Re-lock station when user returns
  }
}

```

6.5.4 `_closeSession()` — App Close Path

```

void _closeSession() {
  _mqtt.stopExperiment(); // ALWAYS send stop, even if session not
  active
  if (_sessionActive) {
    if (_sessionDocId != null) {
      _firestore.endSession(_sessionDocId!, _peakCurrentMa);
      _sessionDocId = null;
    }
    _sessionActive = false;
  }
  if (_myStation != null && !_lockReleased) {
    _lockReleased = true;
    _firestore.deleteStationLock(widget.experiment.id, _myStation!.id);
    // Deletes the lock fields entirely – station is immediately
    available
  }
}

```

6.5.5 `_exitExperiment()` — Back Button Path

```

Future<void> _exitExperiment() async {
  await _stopSession();
  if (_myStation != null && !_lockReleased) {
    _lockReleased = true;
    await _firestore.unlockStation(widget.experiment.id, _myStation!.id);
    // Sets lock fields to null – station is available again
  }
}

```

6.6 MainActivity.kt — Force Kill on Swipe-Away

```

package com.remotelab.experimenthome
import android.os.Process
import io.flutter.embedding.android.FlutterActivity

class MainActivity : FlutterActivity() {
  override fun onDestroy() {
    super.onDestroy()
    if (isFinishing) {
      Process.killProcess(Process.myPid())
    }
  }
}

```

WHY THIS IS NEEDED: When an Android user swipes the app away from the recent apps list, Android does not always immediately kill the process. The activity may be destroyed (`onDestroy` called) but the process lingers in the background. This means the

Flutter Dart isolate may still be running, and new MQTT reconnections could interfere with the next session. `Process.killProcess()` ensures the entire process is terminated when the activity is finishing (i.e., when it is being intentionally closed, not just rotated or backgrounded). The `isFinishing` check prevents killing the process on legitimate non-destructive lifecycle events.

6.7 Catalog Screen — Station Availability Logic

```
// Availability indicator logic (catalog_screen.dart):  
// total = experiment.totalStations  
// available = experiment.availableStations (stations with isAvailable ==  
true)  
  
if (total == 0):           Gray dot    - "Offline"  
if (available > 0):       Green dot    - "X of Y stations available"  
if (total > 0 &&  
    available == 0):      Orange dot   - "In use"  
  
// The card is TAPPABLE even when "In use".  
// This allows the user to enter the screen, which triggers _init()  
// and automatically clears orphaned/stale locks.
```

7. KEY DESIGN DECISIONS AND RATIONALE

This section explains the "why" behind the important design choices made in ExperimentHome. These are non-obvious decisions that required analysis and iteration.

7.1 Station Locking: `FieldValue.delete()` vs Setting to Null

When releasing a station lock on app close, the lock fields are DELETED from Firestore (using `FieldValue.delete()`) rather than set to null. When the back button is pressed, the fields are set to null.

REASON: On app close, we are in a time-critical path (the app is being killed). Using `FieldValue.delete()` is a server-side atomic operation that removes the field entirely, leaving no stale data. Setting to null still leaves the field present with a null value, which works but requires the app to be in a stable enough state to write. The delete is the "nuclear option" — it works even if the document was in an inconsistent state. The `Station.isAvailable` getter treats both null and absent fields as "available".

7.2 Stale Lock Detection (10-Minute Threshold)

A station lock is considered "stale" if it was acquired more than 10 minutes ago. Stale locks are treated as available and are cleaned up on the next entry into the experiment screen.

REASON: If a user's app crashes or their internet drops permanently while they hold a lock, the lock will never be released by normal means. The 10-minute threshold means other users are at most blocked for 10 minutes. The threshold was chosen to be long enough to not interrupt normal sessions (experiments typically last 2–5 minutes) but short enough not to be a significant barrier.

7.3 Tracking toReclaim (Not Re-querying Firestore)

In `_init()`, after deleting a stale/orphaned lock, the algorithm uses a `toReclaim` reference (captured before the delete) to claim the station, rather than re-querying Firestore or re-checking `widget.stations`.

REASON: `widget.stations` is a snapshot passed from the catalog screen when the user navigated to `ExperimentScreen`. After the delete, this snapshot still shows the old lock. Re-querying Firestore is possible but introduces a race condition (another user could claim the station between the delete and the query). Using `toReclaim` directly is both faster and race-condition free.

7.4 Always Sending MQTT Stop on App Close

The `_closeSession()` function always publishes `{"action":"stop"}` to the ESP32, regardless of whether the session is active.

REASON: If the session was active (experiment running) and the app closes without sending a stop, the ESP32 would keep the relay closed indefinitely, wasting power and

leaving the circuit energised for the next user. The unconditional stop ensures the ESP32 always returns to a safe state. The extra stop command when no session is active is harmless (ESP32 just calls `openAllRelays()` which is already the current state).

7.5 Active-LOW Relay and Boot-Safe Initialization

The relay module is active-LOW. All relay pins are driven HIGH as the first action in `setup()`, before any other initialization.

REASON: GPIO pins on the ESP32 float (undefined state) during boot. On an active-LOW relay module, a floating pin can be momentarily LOW, closing the relay. This caused the relay to click and the LED to flicker on every power-on. Driving all pins HIGH immediately in `setup()` minimizes this boot-glitch window. The `closeRelay(0)` call that was originally at the end of `setup()` was removed entirely — relays should stay open until an explicit start command is received.

7.6 ACS712 Sensitivity: 100 mV/A (Not 185 mV/A)

The firmware uses a sensitivity constant of 100.0 mV/A for the ACS712.

REASON: The ACS712 comes in three variants: 5A (185 mV/A), 20A (100 mV/A), and 30A (66 mV/A). The project uses the 20A version. Using the 5A sensitivity (185 mV/A) on a 20A module caused all current readings to show zero because the calculation would show a positive current only if the voltage exceeded 2.5V by more than the noise floor — which was being divided by the wrong sensitivity.

7.7 `Process.killProcess()` in `MainActivity.kt`

The Android `MainActivity` overrides `onDestroy()` and calls `Process.killProcess()` when `isFinishing` is `true`.

REASON: Android's process lifecycle does not guarantee that the Flutter Dart isolate terminates immediately when the activity is destroyed on swipe-away. In testing, the process was observed to linger, allowing the MQTT client to reconnect and interfere with subsequent sessions. Force-killing the process ensures clean termination. The `isFinishing` guard ensures this only happens on intentional app termination, not on screen rotations or other lifecycle events.

7.8 Removed Feature: Idle Auto-Stop

An idle auto-stop feature (which stopped the experiment after 5 minutes of inactivity and released the station) was implemented and then removed.

REASON: The feature was found to be unnecessarily disruptive — students conducting measurements or taking notes would lose their session mid-experiment. The station locking mechanism with the 10-minute stale detection provides sufficient protection against abandoned sessions.

8. CHALLENGES ENCOUNTERED AND SOLUTIONS

8.1 Challenge: Station Showed "In Use" After App Was Closed

Symptom:

When the user closed the app (home button or swipe-away) while holding a station lock, the station remained locked in Firestore. On reopening, the station showed "In use" and was not available to any user.

Root Cause:

Root cause: Android does not call `dispose()` when an app is sent to background or swiped away. The Flutter lifecycle events `paused/hidden/detached` are fired, but the initial implementation did not listen for these. The station was only released in `dispose()` (back button), which was never called.

Solution:

Solution: Implemented `WidgetsBindingObserver` in `ExperimentScreen`. The `didChangeAppLifecycleState` callback now calls `_closeSession()` on `paused`, `hidden`, or `detached` states. `_closeSession()` deletes the station lock fields using `FieldValue.delete()`. Additionally, `MainActivity.kt` was modified to call `Process.killProcess()` on `onDestroy()` to ensure the process is fully terminated on swipe-away.

8.2 Challenge: Stale Snapshot Preventing Re-Claim After Lock Cleanup

Symptom:

After `_init()` deleted an orphaned lock from Firestore and then called `_tryClaimStation()`, the station would still appear locked because `_tryClaimStation()` was checking `widget.stations` — a frozen snapshot from when the screen was opened — which still showed the lock.

Root Cause:

Root cause: Firestore listeners are asynchronous. The delete is sent to Firestore, but `widget.stations` (passed from the catalog) is not updated. Checking `widget.stations` immediately after the delete still shows the old state.

Solution:

Solution: Added a `toReclaim` variable in `_init()` that tracks the first station whose lock was deleted. Instead of calling `_tryClaimStation()` (which checks the stale snapshot), the code calls `_claimStation(toReclaim)` directly, bypassing the snapshot entirely.

8.3 Challenge: Kotlin Compile Error — `onTaskRemoved` Not Recognised

Symptom:

Initial attempt to detect app swipe-away used `onTaskRemoved()` in `MainActivity.kt`. The Kotlin compiler rejected it: "'onTaskRemoved' overrides nothing".

Root Cause:

Root cause: `onTaskRemoved()` is defined on `Service` (Android's background service class), not on `Activity`. `FlutterActivity` extends `Activity`, not `Service`.

Solution:

Solution: Switched to `onDestroy()` with an `isFinishing` check. `onDestroy()` is called on Activity destruction (which includes swipe-away). The `isFinishing` flag distinguishes intentional destruction from configuration changes (e.g., screen rotation).

8.4 Challenge: Build Failed — Not Enough Storage on Test Device**Symptom:**

Running `flutter run -d RFCR71S4HLM` failed with: "android.os.ParcelableException: java.io.IOException: Requested internal only, but not enough space".

Root Cause:

Root cause: A secondary test device (A002SH) had insufficient internal storage. Flutter was attempting to install on this device instead of the Samsung.

Solution:

Solution: Explicitly targeted the Samsung device: `flutter run -d RFCR71S4HLM`. Added this device ID to the development workflow.

8.5 Challenge: Relay Clicking and LED Flickering at Boot**Symptom:**

Every time the ESP32 was powered on, the relay clicked multiple times and the LED connected to it blinked before settling.

Root Cause:

Root cause 1: GPIO pins float (undefined) during the ESP32 boot sequence, before `setup()` runs. On an active-LOW relay module, floating pins can be momentarily LOW, closing the relay.

Root cause 2: The original firmware had a `closeRelay(0)` call at the end of `setup()`, which deliberately closed relay 1 as a "default" state.

Solution:

Solution: All relay pins are now driven HIGH as the FIRST action in `setup()`. The `closeRelay(0)` call was removed. All relays remain open until the user sends an explicit start command via MQTT.

8.6 Challenge: Current Reading Always Zero**Symptom:**

The app always showed 0 mA for the current reading, even with the relay closed and the circuit complete.

Root Cause:

Root cause: The firmware was using `ACS712_SENSITIVITY_MV_PER_A = 185.0`, which is the sensitivity for the 5A version of the ACS712. The project uses the 20A version, which has sensitivity = 100.0 mV/A. With the wrong constant, the calculated current change was too small to exceed the ADC noise floor.

Solution:

Solution: Changed the sensitivity constant to 100.0 mV/A. Also verified the ACS712 VCC and GND wiring. Note: The ACS712-20A is fundamentally poorly suited for measuring currents of 3–50 mA (its intended range is 1–20 A). The voltage change at 50 mA is only 5 mV, within the ESP32 ADC noise floor.

9. END-TO-END EXPERIMENT WORKFLOW

This section describes the complete step-by-step process a student follows to conduct an Ohm's Law experiment using ExperimentHome.

Step 1: Hardware Power-On

The lab technician powers on the ESP32 station. The ESP32 boots, drives all relay pins HIGH (all relays open, LED off), connects to Wi-Fi ("clinton"), connects to HiveMQ broker via TLS, registers the LWT message, subscribes to the control topic, and turns the blue LED on (fully connected indicator). The ESP32 now publishes a heartbeat every 5 seconds.

Step 2: App Launch and Login

The student opens ExperimentHome on their smartphone. If not logged in, they are presented with the login screen. They enter their email and password. Firebase Authentication validates the credentials and returns a user session.

Step 3: Experiment Catalog

The catalog screen loads and queries Firestore for the list of experiments. The Ohm's Law experiment shows its station availability: if the station lock fields are absent or null, a green dot shows "1 of 1 stations available". If locked by another user, an orange dot shows "In use". The ESP32 heartbeats are received by the app (via MQTT) keeping the online status indicator up to date.

Step 4: Claiming a Station

The student taps the Ohm's Law experiment card. ExperimentScreen opens and `_init()` runs. It checks `widget.stations` for orphaned/stale locks and clears any found. It then writes the station lock to Firestore: `{lockedBy: UID, lockedByName: "Name", lockedSince: <timestamp>}`. Other users' catalog screens update via Firestore real-time listener to show "In use". The app then connects to MQTT and subscribes to the current and status topics.

Step 5: Selecting a Resistor

The resistor selector widget shows four options: 100Ω, 220Ω, 470Ω, 1000Ω. The student taps 220Ω. The app publishes: `{"action": "select_resistor", "value": 220}` to the control topic. The ESP32 receives this, finds relay index 1 (220Ω → relay 2 → GPIO 26), stores `activeRelayIndex = 1`. The relay does NOT close yet (experiment not started).

Step 6: Starting the Experiment

The student taps "Start Experiment". The app creates a session record in Firestore (`resistorOhms: 220, startedAt: now`) and publishes `{"action": "start"}` to the control topic. The ESP32 sets `experimentRunning = true` and calls `closeRelay(1)`, driving GPIO 26 LOW. Relay 2 closes. Current flows: 5V → ACS712 → Relay 2 COM → NO → 220Ω → LED → GND. The LED lights up.

Step 7: Live Data

Every 1 second, the ESP32 calls `publishCurrent()`. `readCurrentMa()` averages 100 ADC samples from GPIO 34 (ACS712), converts to mA, and publishes: `{"current_mA":22.7,"timestamp":1234}`. The app receives this via MQTT subscription, adds it to the readings list (max 120 entries = 2 minutes), updates the `CurrentDisplay` widget and the `LiveGraph` scrolling chart.

Step 8: Changing Resistor Mid-Experiment

The student taps 100Ω. The app publishes `{"action":"select_resistor","value":100}`. ESP32 opens all relays (`openAllRelays()`), then closes relay 1 (GPIO 25 LOW). Current now flows through 100Ω. Current increases from ~22.7mA to ~50mA. The graph shows a step change. Ohm's Law is demonstrated: $I = V/R$.

Step 9: Stopping the Experiment

The student taps "Stop Experiment". The app publishes `{"action":"stop"}`. ESP32 sets `experimentRunning = false`, calls `openAllRelays()` (all GPIO HIGH). All relays open, LED turns off. The app calls `_firestore.endSession()` to write `peakCurrentMa` and `endedAt` to the session record.

Step 10: Exiting

Student presses the back button. `_exitExperiment()` is called. `unlockStation()` sets `lockedBy/lockedByName/lockedSince` to null in Firestore. MQTT is disconnected. The catalog screen now shows the station as available (green dot).

Step 11: Abnormal Exit (Swipe Away)

If instead the student swipes the app away: Android calls `onDestroy()` in `MainActivity`. Flutter `WidgetsBindingObserver` fires `didChangeAppLifecycleState` with `detached/paused` before process kill. `_closeSession()` is called: publishes stop to MQTT, calls `deleteStationLock()` (`FieldValue.delete()` removes lock fields). `Process.killProcess()` terminates the process. Station is immediately available in Firestore.

10. CORE ALGORITHMS (PSEUDOCODE)

10.1 Station Claim Algorithm

```
ALGORITHM ClaimStation
INPUT:  widget.stations (snapshot), currentUser
OUTPUT: claimed Station or null
BEGIN
  toReclaim ← null
  FOR EACH station IN widget.stations DO
    IF station.lockedBy = currentUser.uid OR station.isStale THEN
      DELETE lock fields from Firestore (FieldValue.delete)
      IF toReclaim = null THEN toReclaim ← station END IF
    END IF
  END FOR

  IF toReclaim ≠ null THEN
    WRITE lock to Firestore for toReclaim // Bypass stale snapshot
    RETURN toReclaim
  END IF

  FOR EACH station IN widget.stations DO
    IF station.lockedBy = null THEN
      WRITE lock to Firestore for station
      RETURN station
    END IF
  END FOR

  RETURN null // All stations in use
END
```

10.2 ACS712 Current Reading Algorithm (ESP32)

```
ALGORITHM ReadCurrentMa
CONSTANTS: ADC_MAX=4095, VCC=3.3, V_ZERO=2.5, SENSITIVITY=0.100, N=100
BEGIN
  sum ← 0
  FOR i ← 1 TO N DO
    sum ← sum + analogRead(GPIO_34)
    WAIT 100 microseconds
  END FOR
  avgADC ← sum / N
  voltage ← (avgADC / ADC_MAX) × VCC
  currentA ← (voltage - V_ZERO) / SENSITIVITY
  currentMa ← currentA × 1000
  RETURN MAX(0, currentMa) // Clamp negative noise to zero
END
```

10.3 Relay Switching Algorithm (ESP32)

```
ALGORITHM SwitchRelay
CONSTANTS: RELAY_PINS = [25, 26, 27, 14], HIGH=open, LOW=closed

PROCEDURE OpenAllRelays:
  FOR i ← 0 TO 3 DO SET GPIO(RELAY_PINS[i]) ← HIGH END FOR

PROCEDURE CloseRelay(index):
  OpenAllRelays() // Ensure only one relay at a time
  SET GPIO(RELAY_PINS[index]) ← LOW // Close target relay
```

10.4 MQTT Command Dispatch Algorithm (ESP32)

```
ALGORITHM OnMqttMessage
INPUT: topic, payload
STATE: experimentRunning, activeRelayIndex
BEGIN
  doc ← ParseJSON(payload)
  SWITCH doc["action"]:
    CASE "start":
      experimentRunning ← TRUE
      CloseRelay(activeRelayIndex)
    CASE "stop":
      experimentRunning ← FALSE
      OpenAllRelays()
    CASE "select_resistor":
      value ← doc["value"]
      FOR i ← 0 TO 3 DO
        IF RESISTOR_VALUES[i] = value THEN
          activeRelayIndex ← i
          IF experimentRunning THEN CloseRelay(i) END IF
          BREAK
        END IF
      END FOR
    END SWITCH
END
```

10.5 Session Lifecycle Algorithm (Flutter App)

```
ALGORITHM SessionLifecycle
STATE: sessionActive, sessionDocId, myStation, lockReleased

PROCEDURE StartSession:
  sessionDocId ← Firestore.saveSession(experimentId, stationId,
resistorOhms)
  MQTT.publish(control, {"action":"start"})
  MQTT.publish(control,
{"action":"select_resistor","value":selectedResistor})
  sessionActive ← TRUE

PROCEDURE StopSession:
  MQTT.publish(control, {"action":"stop"})
  IF sessionDocId ≠ null THEN
    Firestore.endSession(sessionDocId, peakCurrentMa)
    sessionDocId ← null
  END IF
  sessionActive ← FALSE

PROCEDURE ExitExperiment:           // Back button
  StopSession()
  IF myStation ≠ null AND NOT lockReleased THEN
    lockReleased ← TRUE
    Firestore.set({lockedBy:null, lockedByName:null, lockedSince:null})
  END IF

PROCEDURE CloseSession:           // App close / background
  MQTT.publish(control, {"action":"stop"}) // Always unconditional
  IF sessionActive THEN StopSession() END IF
  IF myStation ≠ null AND NOT lockReleased THEN
    lockReleased ← TRUE
    Firestore.update({lockedBy: DELETE, lockedByName: DELETE,
```

```
lockedSince: DELETE})  
END IF
```

11. REQUIREMENTS SUMMARY

11.1 Functional Requirements

ID	Requirement	Implemented?
FR-01	User registration and email/password authentication	Yes — Firebase Auth
FR-02	Experiment catalog with real-time availability display	Yes — Firestore listener
FR-03	Station availability: Available / In use / Offline indicators	Yes — catalog_screen.dart
FR-04	Station locking (one user at a time)	Yes — Firestore lock fields
FR-05	Automatic lock release on app close/home/swipe-away	Yes — WidgetsBindingObserver + onDestroy()
FR-06	Stale lock cleanup (>10 min old locks reclaimed)	Yes — _init() in ExperimentScreen
FR-07	Remote experiment start/stop	Yes — MQTT "start"/"stop" commands
FR-08	Resistor selection (100/220/470/1000 Ω)	Yes — MQTT "select_resistor" command
FR-09	Real-time current display (1 reading/second)	Yes — MQTT current topic
FR-10	Live scrolling current graph (last 2 minutes)	Yes — live_graph.dart widget
FR-11	Session recording (start, resistor, peak current, end)	Yes — Firestore sessions collection
FR-12	Session history view	Yes — session history screen
FR-13	ESP32 online/offline indicator in app	Yes — heartbeat + LWT via MQTT
FR-14	MQTT stop command sent unconditionally on app close	Yes — _closeSession()

Table 11.1: Functional Requirements Implementation Status

11.2 Non-Functional Requirements

ID	Category	Requirement	Met?
NFR-01	Performance	End-to-end command latency < 2 seconds	Yes — MQTT typical latency < 200ms
NFR-02	Real-time	Current readings at ≥ 1 /second	Yes — 1s publish interval in firmware
NFR-03	Security	All MQTT traffic encrypted (TLS 1.2)	Yes — port 8883, wifiClient.setInsecure() for dev
NFR-04	Security	User auth via Firebase (no custom server)	Yes — Firebase Auth
NFR-05	Reliability	ESP32 auto-reconnects on MQTT disconnect	Yes — reconnect loop in loop()
NFR-06	Reliability	Station always released on abnormal close	Yes — WidgetsBindingObserver + Process.killProcess()
NFR-07	Cross-platform	Android and iOS from single codebase	Yes — Flutter
NFR-08	Usability	Core experiment controls within 2 taps	Yes — all controls on single screen
NFR-	Scalability	Architecture supports multiple	Yes — topic structure

09		experiments	parameterized
----	--	-------------	---------------

Table 11.2: Non-Functional Requirements

12. TESTING AND RESULTS

12.1 Test Environment

Mobile Device: Samsung SM-G781U (RFCR71S4HLM), Android 13
Test connection: Wi-Fi (local network, connected to same router as ESP32)
Firebase project: Production Firestore and Authentication
MQTT broker: HiveMQ Cloud (production instance)
Test accounts: Multiple test accounts created for concurrent-user testing

12.2 Test Cases and Results

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Login with valid credentials	Navigate to catalog screen	Navigated to catalog	PASS
TC-02	Login with wrong password	Error message shown	Error shown, stay on login	PASS
TC-03	View experiment when station available	Green dot, "1 of 1 available"	Green dot displayed	PASS
TC-04	Tap experiment → claim station	Lock written to Firestore, MQTT connected	Lock confirmed in Firestore console	PASS
TC-05	Select 100Ω → Start → Check relay	Relay 1 (GPIO 25) closes	Relay click heard, LED lights up	PASS
TC-06	Change resistor while running	Relay switches, current changes	Relay switches, LED brightness changes	PASS
TC-07	Live graph updates	Graph scrolls with new readings every 1s	Graph updates observed	PASS
TC-08	Stop experiment	All relays open, LED off, session saved	Relays open, LED off, Firestore updated	PASS
TC-09	Press back button	Lock fields set to null in Firestore	Fields set to null (confirmed in Firestore console)	PASS
TC-10	Swipe app away	Lock fields deleted from Firestore within seconds	Fields deleted (confirmed in Firestore console)	PASS
TC-11	Reopen app after swipe	Station available, can claim again	Station shown as available, claimed successfully	PASS
TC-12	User A holds lock; User B checks catalog	User B sees "In use"	"In use" shown to User B in real time	PASS
TC-13	User A closes app; User B checks	User B sees station available	Station available within ~2 seconds	PASS
TC-14	ESP32 powered off while session active	App shows offline indicator	Device status shows offline after heartbeat timeout	PASS
TC-15	Boot ESP32: relays stay open	No relay clicking, LED stays off	No clicking or blinking observed	PASS

Table 12.1: Test Cases and Results

12.3 Known Limitations

- ACS712-20A sensor: The 20A sensor has poor sensitivity at the milliamp range (3–50 mA). Current readings are noisy and may not precisely match theoretical Ohm's Law values. The proportional relationship (larger $R \rightarrow$ smaller I) is still observable. Fix: Replace with ACS712-05A or INA219 for precise mA measurements.
- TLS verification: The ESP32 firmware uses `wifiClient.setInsecure()`, which skips TLS certificate verification. For a production deployment, a proper CA certificate should be embedded in the firmware.
- Single station: Only one hardware station is implemented. The architecture supports multiple stations (parameterized topic structure and Firestore path), but physical hardware for a second station was not built.
- Wi-Fi credentials hardcoded: The Wi-Fi SSID and password are hardcoded in the firmware. A production system would use a provisioning mechanism (e.g., BLE provisioning).

13. CONTEXT AND GUIDANCE FOR REPORT WRITING

13.1 Dissertation Report Structure (University of Buea Format)

Chapter	Title	Key Content
Chapter 1 (10-15 pages)	Introduction	Problem statement, objectives, scope, motivation, organization of report. Focus: Why remote labs matter; What this project does; Who benefits.
Chapter 2 (15-20 pages)	Literature Review	Review of existing remote lab systems (iLab, VISIR, LabsLand, etc.), IoT frameworks, MQTT protocol, mobile app frameworks, Firebase. Compare and contrast with ExperimentHome approach.
Chapter 3 (15-25 pages)	Analysis and Design	Methodology, requirements, use cases, architecture, data model, algorithms, resolution process. Use content from Sections 4–10 of this document.
Chapter 4 (15-20 pages)	Implementation	Actual code implementation details, Flutter app screens, ESP32 firmware, Firebase configuration, MQTT setup. Use content from Sections 5–6.
Chapter 5 (10-15 pages)	Testing and Results	Test cases, test results, screenshots, current measurements, Ohm's Law verification table, analysis. Use content from Section 12.
Chapter 6 (5-10 pages)	Conclusion	Summary of work done, objectives achieved, limitations, future work recommendations.

Table 13.1: Dissertation Report Structure

13.2 Key Narrative Points for the Report

- The system WORKS end-to-end: A student can log in, claim a station, select a resistor, start the experiment, see live current readings update every second, and see the LED physically switch on in the lab. The back button correctly releases the station, and swipe-away also correctly releases the station.
- The station locking mechanism is the most technically challenging aspect. It required multiple iterations to correctly handle Android process lifecycle edge cases (paused/hidden/detached states, process not terminating on swipe-away).
- Ohm's Law is demonstrated qualitatively: The LED visibly changes brightness as the resistor changes, and the current reading decreases as resistance increases. Precise mA measurements are limited by the ACS712-20A sensor choice.
- The architecture is designed for scalability: Adding a new experiment type requires only a new Firestore document and new MQTT topic subtree. Adding a new station for an existing experiment requires only adding a station document to Firestore and deploying another ESP32.
- The project demonstrates integration of four distinct technology areas: embedded systems (ESP32/Arduino), IoT communication (MQTT/TLS), cloud backend (Firebase), and mobile application development (Flutter). This breadth is a key strength to emphasize in the report.

13.3 Ohm's Law Theory to Include in Report

Ohm's Law states that the current (I) flowing through a conductor is directly proportional to the voltage (V) applied across it and inversely proportional to its resistance (R), provided the temperature remains constant. Mathematically:

$$I = V / R$$

Where: I = current in Amperes (A), V = voltage in Volts (V), R = resistance in Ohms (Ω).

In this experiment, the supply voltage is V = 5V (constant). Four different resistors are used: 100 Ω , 220 Ω , 470 Ω , and 1000 Ω . The expected currents are: 50 mA (100 Ω), 22.7 mA (220 Ω), 10.6 mA (470 Ω), and 5 mA (1000 Ω). The student switches between resistors using the app and observes the current change, verifying the inverse relationship between R and I.

13.4 Student and Institution Details for Report Header

Title:	Design and Implementation of a Cross-Platform Mobile Application
	for Monitoring and Controlling Remote Lab Experiments
Student:	Ekane Clinton Ntongwe
Matricule:	FE22A198
Email:	clintonyayyi9@gmail.com
Institution:	University of Buea
Faculty:	Faculty of Engineering and Technology (FET)
Department:	Department of Computer Engineering
Level:	400 Level (Final Year)
Degree:	Bachelor of Engineering in Computer Engineering
Year:	2026